UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ingeniería en Tecnologías de la Información e Innovación Digital

ACTIVIDAD 6
Algoritmo genético para distribución

Datos de entrega	Información
Asignatura	Fundamentos de Inteligencia Artificial
Integrantes	Bryan Salvador Ramos López · Alexis Kevin Pérez García · Jorge Lidio Barragán Villanueva · Emmanuel Guzmán García
Grupo	172
Docente	Mtro. Javier García Lara
Fecha indicada en el enunciado	26 de octubre de 2026

Trabajo en equipo versionado en GitHub. Se incluyen planteamiento, procedimiento, evidencia, conclusiones y referencias.

Caso y formulación

Se simula una brigada de entrega de materiales educativos en Atlautla: sale de una base, visita ocho puntos y regresa. Las calles son datos reales de OpenStreetMap y los puntos de entrega son hipotéticos, seleccionados con semilla fija. No se describen entregas, clientes ni direcciones reales.

Cada cromosoma es una permutación de los índices 1 a 8; la base ocupa el índice 0 y se añade al inicio y final del circuito. La aptitud a minimizar es la suma de las distancias viales mínimas entre paradas consecutivas. La matriz se precalcula con Dijkstra sobre la red incluida en Act5.

Implementación con DEAP

Parámetro	Valor
Población	80
Generaciones	30
Semilla	42
Selección	torneo de 3
Cruce	cxOrdered (p=0.85)
Mutación	mutShuffleIndexes (p=0.25; gen=0.12)

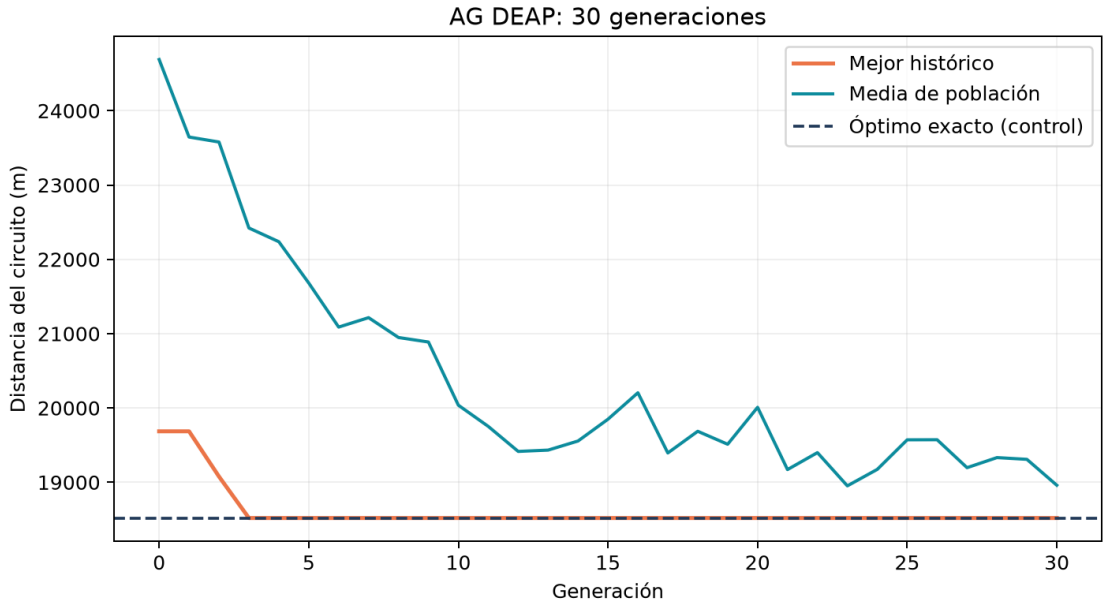
Parámetro	Valor
Elitismo	2

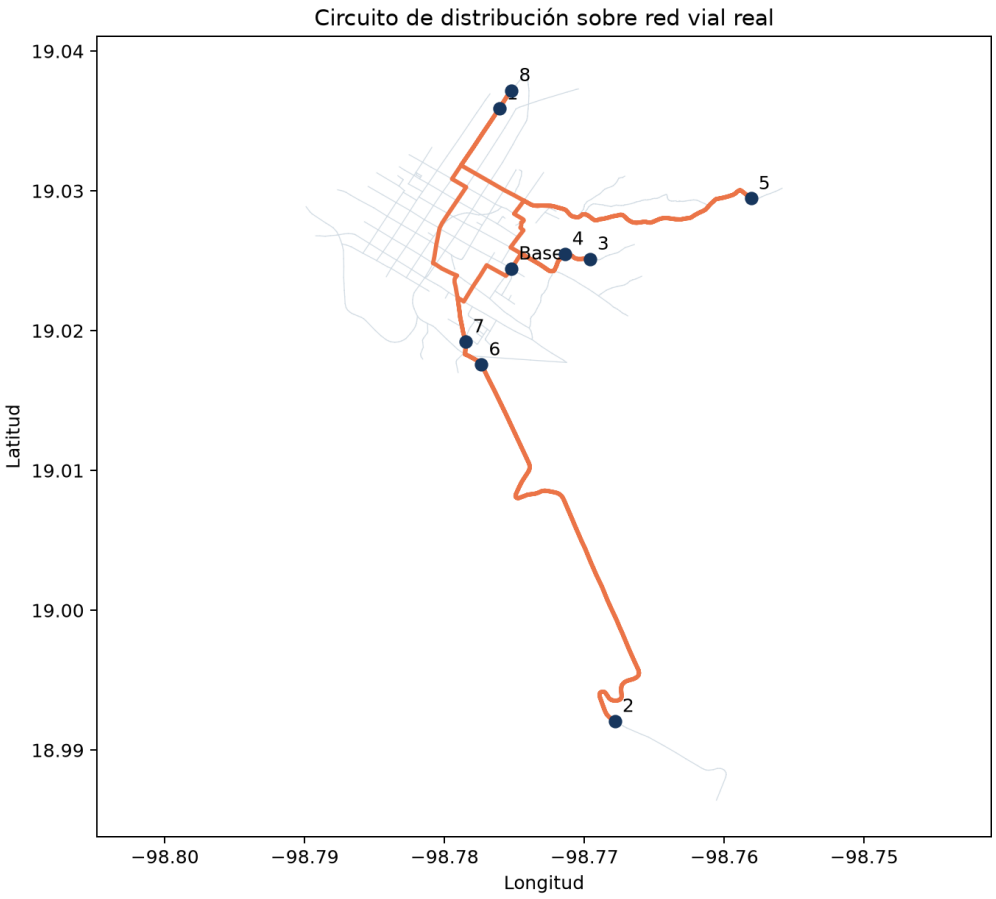
Procedimiento: generar 80 permutaciones, evaluar costos, conservar dos élites, seleccionar por torneo de tres, cruzar con operador ordenado, mutar permutando posiciones y repetir 30 generaciones. El mejor histórico y la media se registran desde la generación 0. El cruce conserva la validez de las permutaciones.

Resultados y control

Método	Distancia del circuito
Algoritmo genético DEAP	18519.580 m
Vecino más cercano	21089.808 m
Óptimo exacto Held-Karp	18519.580 m
Brecha del AG contra óptimo	0.00 %

Orden del AG: Base → 7 → 6 → 2 → 8 → 1 → 5 → 3 → 4 → Base. Held-Karp se utilizó como control exacto para esta instancia pequeña, no como componente del algoritmo genético. Una coincidencia con el óptimo en esta semilla no garantiza que el AG lo alcance en toda instancia.





Reproducción y límites

Instalar `pip install -r requirements.txt` y ejecutar `python Act6/act6_genetico.py` desde la raíz. El archivo **Act6/resultados/act6_resultados.json** conserva matriz, parámetros, ruta e historia completos. La interfaz HTML permite repetir una simulación genética en el navegador; la implementación evaluada usa DEAP.

Conclusión

La búsqueda evolutiva redujo el recorrido frente a vecino más cercano y alcanzó el costo exacto de control en este ejemplo. Para adoptar el modelo en una empresa o programa social se necesitarían horarios, capacidad, costos de operación, restricciones reales y protección de datos.

Referencias (APA 7.ª ed.)

Fortin, F.-A., De Rainville, F.-M., Gardner, M.-A., Parizeau, M., & Gagné, C. (2012). DEAP: Evolutionary algorithms made easy. Journal of Machine Learning Research, 13, 2171–2175.

OpenStreetMap contributors. (2026). OpenStreetMap [datos geográficos]. https://www.openstreetmap.org/copyright

Wirsansky, E. (2020). Hands-on genetic algorithms with Python. Packt Publishing.

Uso de inteligencia artificial y verificación

Se utilizó ChatGPT/Codex para apoyar la organización del reporte, la redacción, el desarrollo de código y gráficos. Se ejecutaron los programas y pruebas locales, y se cotejaron las afirmaciones documentales con las fuentes citadas. Quienes presentan el trabajo deben revisarlo, comprenderlo y poder explicarlo.

Anexo: código Python completo

Archivo ejecutable **Act6/act6_genetico.py**. La numeración facilita la revisión; el archivo del repositorio es la versión directamente ejecutable.

```
001  """Algoritmo genético DEAP para una ruta de distribución educativa en Atlautla.
002
003  Los ocho puntos de entrega son hipotéticos, elegidos sobre nodos reales de OSM.
004  La matriz de costos usa caminos mínimos de la red vial incluida en Act5.
005  """
006  from __future__ import annotations
007
008  import json
009  import math
010  import random
011  import statistics
012  from pathlib import Path
013
014  import matplotlib
015  matplotlib.use("Agg")
016  import matplotlib.pyplot as plt
017  import networkx as nx
018  from deap import base, creator, tools
019
020  import sys
021  sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "Act5"))
022  from act5_rutas import cargar_red, extremos, punto
023
024  BASE = Path(__file__).resolve().parent
025  OUT = BASE / "resultados"
026  SEMILLA = 42
027  POBLACION = 80
028  GENERACIONES = 30
029  PROB_CRUCE = .85
030  PROB_MUTACION = .25
031  PARADAS = 8
032
033
034  def elegir_paradas(g: nx.Graph, deposito: str) -> list[str]:
035      """Ocho puntos hipotéticos elegidos con semilla fija sobre la red real."""
036      candidatos = sorted(n for n in g.nodes if n != deposito)
037      return [deposito, *random.Random(2026).sample(candidatos, PARADAS)]
038
039
040  def matriz_distancias(g: nx.Graph, puntos: list[str]) -> list[list[float]]:
041      return [[round(nx.shortest_path_length(g, a, b, weight="weight"), 3)
042               for b in puntos] for a in puntos]
043
044
045  def longitud(cromosoma: list[int], matriz: list[list[float]]) -> float:
046      """Costo del ciclo depósito -> ocho puntos -> depósito, en metros."""
047      if sorted(cromosoma) != list(range(1, len(matriz))):
048          raise ValueError("El cromosoma debe ser una permutación de las paradas 1..8")
049      visita = [0, *cromosoma, 0]
050      return sum(matriz[a][b] for a, b in zip(visita, visita[1:]))
051
052
053  def vecino_cercano(matriz: list[list[float]]) -> list[int]:
054      pendiente = set(range(1, len(matriz)))
055      ruta = []
056      actual = 0
057      while pendiente:
058          siguiente = min(pendiente, key=lambda n: (matriz[actual][n], n))
059          ruta.append(siguiente)
060          pendiente.remove(siguiente)
061          actual = siguiente
062      return ruta
063
064
065  def optimo_exacto(matriz: list[list[float]]) -> tuple[float, list[int]]:
066      """Held-Karp para ocho paradas; referencia de validación, no parte del AG."""
067      n = len(matriz) - 1
068      dp = {}
069      for j in range(1, n+1):
070          dp[(1 << (j-1), j)] = (matriz[0][j], [j])
071      for mask in range(1, 1 << n):
072          for j in range(1, n+1):
073              if not mask & (1 << (j-1)) or mask == 1 << (j-1):
074                  continue
075              previo = mask ^ (1 << (j-1))
076              candidatos = [(dp[(previo,k)][0] + matriz[k][j], dp[(previo,k)][1] + [j])
077                            for k in range(1,n+1) if previo & (1 << (k-1))]
078              dp[(mask,j)] = min(candidatos, key=lambda x: x[0])
079      return min(((dp[((1 << n)-1,j)][0] + matriz[j][0], dp[((1 << n)-1,j)][1])
080                  for j in range(1,n+1)), key=lambda x:x[0])
```

```python


def cruce_ordenado(a: list[int], b: list[int]) -> tuple[list[int],list[int]]:
    """DEAP cxOrdered indexa desde cero; los índices externos son 1..8."""
    copia_a, copia_b = [v-1 for v in a], [v-1 for v in b]
    tools.cxOrdered(copia_a, copia_b)
    a[:], b[:] = [v+1 for v in copia_a], [v+1 for v in copia_b]
    return a, b


def algoritmo_genetico(matriz: list[list[float]], semilla: int = SEMILLA) -> dict:
    random.seed(semilla)
    if not hasattr(creator, "FitnessMinFIA"):
        creator.create("FitnessMinFIA", base.Fitness, weights=(-1.0,))
    if not hasattr(creator, "RutaFIA"):
        creator.create("RutaFIA", list, fitness=creator.FitnessMinFIA)
    toolbox = base.Toolbox()
    toolbox.register("indices", random.sample, range(1, len(matriz)), len(matriz)-1)
    toolbox.register("individual", tools.initIterate, creator.RutaFIA, toolbox.indices)
    toolbox.register("population", tools.initRepeat, list, toolbox.individual)
    toolbox.register("evaluate", lambda ind: (longitud(ind, matriz),))
    toolbox.register("mate", cruce_ordenado)
    toolbox.register("mutate", tools.mutShuffleIndexes, indpb=.12)
    toolbox.register("select", tools.selTournament, tournsize=3)
    poblacion = toolbox.population(n=POBLACION)
    for ind in poblacion:
        ind.fitness.values = toolbox.evaluate(ind)
    hof = tools.HallOfFame(1)
    hof.update(poblacion)
    historia = [{"generacion":0,
                 "mejor_m":round(hof[0].fitness.values[0],3),
                 "media_m":round(statistics.mean(i.fitness.values[0] for i in poblacion),3)}]
    for gen in range(1, GENERACIONES+1):
        elite = [toolbox.clone(x) for x in tools.selBest(poblacion, 2)]
        hijos = [toolbox.clone(x) for x in toolbox.select(poblacion, POBLACION-2)]
        for a,b in zip(hijos[::2], hijos[1::2]):
            if random.random() < PROB_CRUCE:
                toolbox.mate(a,b)
                del a.fitness.values, b.fitness.values
        for ind in hijos:
            if random.random() < PROB_MUTACION:
                toolbox.mutate(ind)
                if ind.fitness.valid:
                    del ind.fitness.values
        for ind in hijos:
            if not ind.fitness.valid:
                ind.fitness.values = toolbox.evaluate(ind)
        poblacion = elite + hijos
        hof.update(poblacion)
        historia.append({"generacion":gen,
                         "mejor_m":round(hof[0].fitness.values[0],3),
                         "media_m":round(statistics.mean(i.fitness.values[0] for i in poblacion),3)})
    return {"ruta":list(hof[0]), "distancia_m":round(hof[0].fitness.values[0],3),
            "historia":historia}


def trazo_completo(g: nx.Graph, puntos: list[str], orden: list[int]) -> list[str]:
    paradas = [0, *orden, 0]
    ruta = []
    for a,b in zip(paradas,paradas[1:]):
        tramo = nx.shortest_path(g,puntos[a],puntos[b],weight="weight")
        ruta.extend(tramo if not ruta else tramo[1:])
    return ruta


def graficar(g: nx.Graph, puntos: list[str], resultado: dict, exacto: float) -> None:
    OUT.mkdir(exist_ok=True)
    h = resultado["historia"]
    fig,ax=plt.subplots(figsize=(8,4.5))
    ax.plot([x["generacion"] for x in h],[x["mejor_m"] for x in h],
            color="#eb7447",lw=2,label="Mejor histórico")
    ax.plot([x["generacion"] for x in h],[x["media_m"] for x in h],
            color="#0d8c9d",lw=1.6,label="Media de población")
    ax.axhline(exacto,color="#223a59",ls="--",label="Óptimo exacto (control)")
    ax.set(xlabel="Generación",ylabel="Distancia del circuito (m)",
           title="AG DEAP: 30 generaciones")
    ax.legend(); ax.grid(alpha=.2); fig.tight_layout()
    fig.savefig(OUT/"act6_convergencia.png",dpi=180);plt.close(fig)

    ruta=trazo_completo(g,puntos,resultado["ruta"])
    fig,ax=plt.subplots(figsize=(8,7))
    for a,b in g.edges:
        x,y=punto(g,a),punto(g,b)
        ax.plot([x[1],y[1]],[x[0],y[0]],color="#d2dce4",lw=.5)
    coords=[punto(g,n) for n in ruta]
```

```
    ax.plot([v[1] for v in coords],[v[0] for v in coords],color="#eb7447",lw=2.3)
    for i,n in enumerate(puntos):
        lat,lon=punto(g,n)
        ax.scatter(lon,lat,color="#17365d",s=40,zorder=5)
        ax.annotate("Base" if i==0 else str(i),(lon,lat),xytext=(4,5),textcoords="offset points")
    ax.set(xlabel="Longitud",ylabel="Latitud",title="Circuito de distribución sobre red vial real")
    ax.set_aspect("equal",adjustable="datalim");fig.tight_layout()
    fig.savefig(OUT/"act6_ruta.png",dpi=180);plt.close(fig)


def ejecutar() -> dict:
    g,fuente=cargar_red()
    deposito,_=extremos(g)
    puntos=elegir_paradas(g,deposito)
    matriz=matriz_distancias(g,puntos)
    resultado=algoritmo_genetico(matriz)
    base=vecino_cercano(matriz)
    optimo,orden_optimo=optimo_exacto(matriz)
    graficar(g,puntos,resultado,optimo)
    trazo=trazo_completo(g,puntos,resultado["ruta"])
    datos={"caso":"Brigada hipotética de entrega de materiales educativos en Atlautla",
           "origen_red":fuente["url"],"paradas_hipoteticas":True,
           "semilla_seleccion_paradas":2026,
           "parametros":{"semilla":SEMILLA,"poblacion":POBLACION,
                         "generaciones":GENERACIONES,"prob_cruce":PROB_CRUCE,
                         "prob_mutacion":PROB_MUTACION,"prob_gen_mutacion":.12,
                         "seleccion":"torneo de 3","cruce":"cxOrdered",
                         "mutacion":"mutShuffleIndexes","elitismo":2},
           "puntos":[{"nombre":"Base" if i==0 else f"Punto {i}","id":n,
                      "latlon":punto(g,n)} for i,n in enumerate(puntos)],
           "matriz_distancias_m":matriz,
           "ruta_ga":resultado["ruta"],"distancia_ga_m":resultado["distancia_m"],
           "historia":resultado["historia"],
           "ruta_vecino_cercano":base,"distancia_vecino_cercano_m":round(longitud(base,matriz),3),
           "ruta_optima_control":orden_optimo,"distancia_optima_control_m":round(optimo,3),
           "brecha_optima_pct":round(100*(resultado["distancia_m"]-optimo)/optimo,2),
           "coordenadas_trazo_ga":[punto(g,n) for n in trazo]}
    (OUT/"act6_resultados.json").write_text(json.dumps(datos,ensure_ascii=False,indent=2),encoding="utf-8")
    print(json.dumps({"distancia_ga_m":datos["distancia_ga_m"],
                      "distancia_vecino_cercano_m":datos["distancia_vecino_cercano_m"],
                      "distancia_optima_control_m":datos["distancia_optima_control_m"],
                      "brecha_optima_pct":datos["brecha_optima_pct"]},indent=2))
    return datos


if __name__=="__main__":
    ejecutar()
```